Course introduction

1.2x | 6:01

You have already learned how to send requests to the Gemini API, receive text responses, and manage its formal factors. Decisions and context windows. You have also used Google AI Studio to prototype and test your prompts.

In this course, you will move beyond the role of a prompter and actively code for how you will become a systems architect who builds an AI agent. This reading is your high-level orientation. It provides you with the skills and tools required to build autonomous systems and also gives you a comprehensive roadmap for your final project.

The shift from writer to doer

Up until now, you have treated the Large Language Model, or LLM, primarily like a very smart, very well-read writer. You give it a prompt, and it gives you a piece of text in return. This is powerful, but it is also passive. The model can only give you a one-time answer. This capability is undeniably powerful, but it has a distinct limits. In its traditional setup, the model is essentially trapped inside a box. It only knows what it learned during its initial training period, and it is mathematically sealed from the dynamic, ever-changing reality of the outside world.

To visualize this better, imagine that you're in a brilliant genius-level assistant. This assistant has a stationary single desk in a room with no windows or doors. You can slide notes under the door, and the assistant can slide notes back to you. This assistant can read your notes and write incredibly detailed responses. However, it is physically, however, incapable of the assistant is limited to and absolutely cannot do anything else. It cannot look out the window to see the weather, it cannot walk over to the phone to call a friend, and it cannot check its own inbox for new messages. It is entirely dependent on you to be its sensory organ and its messenger. It can only act and interact via the notes you slide under the door.

This is the fundamental limitation of a standard text generation model. It's a profound shift in capability, but it is not a doer. It is not a system. In this course, we are going to make this transition from writer to doer.

Understanding function calling

The key feature is easily create tools every agent is a feature called function calling. Function calling can sound like a complicated, intimidating programming term. But, as we like to software engineering. However, the core concept is actually quite simple. It is a very natural way for the model to ask for help from your specific code.

Think back to the analogy of the assistant locked in the room. Imagine you've taken a specialized speaker system to the assistant's desk. You've also given the assistant a specific instruction: "If you ever need to know the weather, don't guess. Just send a specific code word into the speaker, and I will come over the door and tell the current temperature."

Now, the workflow changes. When you ask the assistant, "Should I wear a raincoat today?" the assistant pauses and thinks. Instead, based on its instructions that to answer this question accurately, it needs current meteorological data. It speaks the specific code word to the speaker.

Example of function calling in a workflow

The architecture of an agent

Building an agent is surprisingly significantly less complicating than writing a simple prompt. In this course, you will learn to act as a systems architect. A robust agent is really composed of three distinct but interconnected components.

First, there is the reasoning engine, which is the brain of the operation. This is the large language model we've introduced earlier. It is the thought maker. It is what's in great use, and how it should behave. You can tell, a meal that would read detailed instructions such as, "You are a helpful, professional assistant. Always check specifically for availability before suggesting a meal. Never assume a time without checking the database."

Second, there are the tools. These are the specific functions you define in your code. You might give your agent a tool called `check_inventory` or `send_email`. Crucially, you must provide the model with a clear description of what your code does so that it knows when and how to use them. If you don't like it, but they've, but a.s. somewhere, the model will try to use this to no avail, and this agent will fail. Precise descriptions are also for success.

Third, there's the orchestration layer. This is the application logic that manages the conversation loop. It handles the back-and-forth between the user, the model, and your tools. It's the code that takes the model's request, manages the loop, You call function the user's input, and this tells, then, it Planner works to call functions, execute the functions, and send the answer back to the. No to success.

Why not use agents for every prompt?

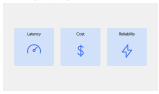
You might wonder, "If agents are so powerful and capable, why do we not use them for every single request?" This is a very important strategic question. While agents are incredibly powerful, they are not always the right choice for every scenario. There are valid reasons to stick with standard prompts for certain tasks.

One major factor is latency. Latency is the time it takes for a system to respond to a user. When an agent has to stop, think, call a tool, wait for the external tool to finish its job, and then process its answers, it takes time. A simple text prompt might take a few seconds, but a complex agent workflow can take minutes. For a high-speed, real-time, or, or even fifteen seconds, this is an exciting or a chaos. And delays or facilities or alarms.

Another factor is cost. Every time the model thinks and sends data back and forth to your code, it uses tokens. A complex agent loop with multiple steps can significantly more tokens than a simple question-and-answer pass. If you are building a system for millions of users, this can scale up very quickly.

Finally, there is reliability. The more moving parts you have in a workflow, the more things can break. If the tool database API is offline, the model can sometimes fix it, even. Even if part is working perfectly. Therefore, part of your job as an engineer is to build robust error handling. Below. You must decide when an agent is sufficiently reliable to be trusted for the tasks it has, complexity, and time.

Considerations for using standard prompts vs. agents



Safety and security in action

Whenever give it models the power to take action in the real world, we must also think deeply about safety. In this course, we will call as a "human in the loop" workflow. This means designing your agent so that it explicitly asks for human permission before taking a sensitive action. For example, the Assist Oversight that the Natural Protocol Agent will not send an email or delete a file without first presenting the proposed action to the user and receiving their explicit confirmation. Always confirm details are correct before I submit the payment."

Your capstone project overview

This course is structured to guide you step by step through building a complete project.

First, you will focus on the tools. You will examine the syntax and mechanics of function calling. You will learn how to define tools and how to handle the data that comes back from them. You will design the architecture for your agent's reasoning and orchestration flow. By the end of this module, you will have a working application running locally on your machine.

Then, you'll shift your focus to production. Building an app that works on your laptop is one thing, getting it on the internet for the scientist use is another challenge entirely. You will look at deployment strategies. You will learn how to containerize code and monitor your app to make sure it stays healthy. You will examine how to deploy your application to the cloud so it can be reliably, 24/7.

Key takeaways

- It's an exciting capability, creating a strong need for adaptability and continuous learning.
- Concepts like function calling and agent architecture are high-demand skills that employers are actively seeking.
- This project serves as an opportunity to build & unique and practical applications.

Now that you understand the AI tools & and how to prompt the model, you have the necessary tools to apply your knowledge and begin building.

Mark as completed

Like Dislike Report an issue